

18-MONTH FULL STACK ENGINEER

MASTER LEARNING ROADMAP

Starting: 11 May 2026 → Target: November 2027

■ Python Advanced	■ Java + Spring Boot	■ React	■ Flutter	■ Go Lang
■ AWS Advanced	■ Docker + K8s	■ AI / ML	■ Trading (Personal)	

■ Daily Hours	3-4 hrs weekdays + 4 hrs weekends
■ Total Duration	18 Months (May 2026 – Nov 2027)
■ Platform	Mac VS Code IntelliJ
■ Target	Full Stack Engineer — Indian + Global Market
■ Resources	Udemy Subscription + Free Resources
■ Certifications	AWS Cloud Practitioner + Solutions Architect
■ Target Salary	■30L–■60L+ / \$120K–\$200K
■ DSA	Daily LeetCode (Easy → Medium → Hard)

Prepared for: Personal Learning | Confidential

■ TABLE OF CONTENTS

PHASE 1 Months 1–3 Python Advanced + Java Basics + Docker

- › Month 1: Python Advanced
- › Month 2: Java + Docker
- › Month 3: Spring Boot Intermediate

PHASE 2 Months 4–6 React Web Frontend

- › Month 4: React Fundamentals
- › Month 5: React + Full Stack
- › Month 6: React Advanced

PHASE 3 Months 7–9 Flutter Mobile Development

- › Month 7: Flutter + Dart
- › Month 8: Flutter Intermediate
- › Month 9: Flutter Advanced

PHASE 4 Months 10–12 Cloud + DevOps (AWS + Kubernetes)

- › Month 10: AWS Fundamentals
- › Month 11: AWS Advanced + K8s
- › Month 12: CI/CD + EKS

PHASE 5 Months 13–14 Go Language + Microservices

- › Month 13: Go Fundamentals
- › Month 14: Go Advanced + Microservices

PHASE 6 Months 15–16 AI/ML Integration

- › Month 15: ML Fundamentals
- › Month 16: Deep Learning + Deployment

PHASE 7 Month 17 AWS Advanced + Solutions Architect Prep

- › Month 17: AWS SA Prep + Certification

PHASE 8 Month 18 Capstone Project + Job Prep

- › Month 18: SmartTrader Pro + Interview Grind

■ SPECIAL SECTIONS

- Revision Checkpoints (every 2 weeks)
- Interview Preparation Checkpoints (months 6, 12, 18)
- LeetCode / DSA — Daily Integration
- Personal Trading Education (parallel track)
- Phase Milestone Banners
- Project Deliverables per Month
- AWS Certification Roadmap
- Final Capstone: SmartTrader Pro Platform

PHASE 1

Python Advanced + Java Basics + Docker

Months 1–3 | 11 May 2026 – 10 Aug 2026

Strengthen Python internals → Learn Java OOP → Spring Boot REST APIs → Docker containerisation

This month you will go from intermediate Python to advanced. You already know the basics — now you will learn the internals that make Python powerful in production. Think of it like upgrading from driving a car to understanding how the engine works.

WEEK 1 — Decorators, Generators, Context Managers | 11–17 May 2026

DAY 1 Python Decorators — Part 1

11 May 2026 (Mon) | ■ 3-4 hrs

■ THEORY — What to Learn

A decorator is a function that wraps another function to extend or change its behaviour WITHOUT modifying the original code. Think of it like adding a phone case to your iPhone — same phone, but now protected. Used everywhere: Flask `@app.route`, FastAPI `@app.get`, Django `@login_required`. Understanding decorators deeply = understanding how frameworks work.

- What is a Higher-Order Function?
 - Functions that take other functions as arguments
 - Or return functions as output
 - Decorators are built on this concept
- Basic Decorator Syntax
 - `@my_decorator` above a function = `my_func = my_decorator(my_func)`
 - The `@` symbol is just syntactic sugar
- Wrapper functions
 - The inner function that wraps the original
 - Must accept `*args`, `**kwargs` to be generic
 - Must return the result of the original function
- Decorator execution order
 - Decorators run at import/definition time, not call time
 - This is a very common interview trap

■ CODE — What to Write

Basic decorator — without `@` syntax first:

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print('Before')
        result = func(*args, **kwargs)
        print('After')
        return result
    return wrapper

# Manual way (no @)
greet = my_decorator(greet)
greet()
```

Same thing with `@` syntax:

```
@my_decorator
def greet():
    print('Hello!')

greet() # Before -> Hello! -> After
```

Timer Decorator — Build This:

```
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'[TIMER] {func.__name__} took {end-start:.4f}s')
        return result
    return wrapper

@timer
def slow():
    time.sleep(1)

slow() # [TIMER] slow took 1.0012s
```

■■ EXERCISES — Hands-on Practice

- ✓ Write a decorator WITHOUT `@` syntax first — then convert to `@`
- ✓ Build a timer decorator that prints function name + time taken
- ✓ Build a logger decorator that prints arguments + return value
- ✓ Build a retry decorator (retries function up to 3 times on exception)
- ✓ Stack all 3 decorators on one function and observe output

■ REAL-WORLD USE CASES

- Flask uses `@app.route()` to register URL handlers
- Django uses `@login_required` to protect views
- FastAPI uses `@app.get()` for API endpoint registration
- pytest uses `@pytest.fixture` for test setup
- Celery uses `@app.task` for async task registration

■ Common Mistakes

- Forgetting to return result from wrapper — function returns `None`
- Not using `*args/**kwargs` — breaks for functions with arguments
- Thinking decorator runs at call time (it runs at definition time)
- Forgetting to return the wrapper function from decorator

■ Best Practices

- Always use `*args, **kwargs` in wrapper for flexibility
- Always return result from wrapper
- Use `functools.wraps` to preserve original function metadata
- Test decorator with multiple function signatures

■ INTERVIEW QUESTIONS

- What is a decorator in Python? Explain without using the word 'decorator'.
- What is the difference between `@decorator` and `decorator(func)`?
- When does a decorator execute — at definition or at call time?
- Why do we use `*args` and `**kwargs` inside the wrapper function?
- What happens if you forget to return result from wrapper?

■ DAILY LEETCODE — Easy

- #1 Two Sum — Easy (use dictionary approach)
- #20 Valid Parentheses — Easy (use stack)

■ GITHUB TASK

- Create repo: `python-advanced-learning` | `mkdir week1` | Add `day1_decorators.py`

■ NOTES TO CREATE

- `notes/week1/decorators_basics.md` — What, Why, How with examples
- Add `@decorator` vs `decorator(func)` comparison table

■ TODAY'S DELIVERABLES

- ✓ File created: `week1/day1_decorators.py`
- ✓ 3 decorators working: `timer`, `logger`, `retry`
- ✓ All decorators tested with different function types
- ✓ Pushed to GitHub with descriptive commit message
- ✓ Notes file created: `notes/week1/decorators_basics.md`

DAY 2

Decorators Part 2 — `functools`, `@property`, Class Decorators

12 May 2026 (Tue) | ■ 3-4 hrs

■ THEORY — What to Learn

Yesterday you learned the basics. Today you go deeper. The problem with basic decorators: they break the function's identity. After decorating, `func.__name__` becomes 'wrapper'. `functools.wraps` fixes this. You'll also learn built-in decorators Python gives you for free (`@property`, `@classmethod`, `@staticmethod`) and class-based decorators.

- `functools.wraps`
 - Preserves `__name__`, `__doc__`, `__module__` of original function
 - Always use it in production decorators
 - `from functools import wraps`
- `@property` decorator
 - Turns a method into a read-only attribute
 - Common in OOP for encapsulation
 - Pair with `@setter` for get/set control
- `@classmethod` vs `@staticmethod`
 - `@classmethod` receives `cls` as first arg (class itself)
 - `@staticmethod` receives no implicit first argument
 - Used for factory methods and utility functions
- Class-based decorators
 - A class with `__call__` method can act as a decorator
 - Useful when decorator needs to maintain state
 - e.g., counting how many times a function was called
- Stacking multiple decorators
 - Applied bottom-up at decoration time
 - `@a @b @c def f() = a(b(c(f)))`

■ CODE — What to Write

`functools.wraps` — always use this:

```
from functools import wraps def my_dec(func): @wraps(func) # preserves metadata def wrapper(*args,
**kwargs): return func(*args, **kwargs) return wrapper
```

`@property` example:

```
class Circle: def __init__(self, radius): self._radius = radius @property def radius(self): return
self._radius @radius.setter def radius(self, value): if value < 0: raise ValueError('Radius cannot be
negative') self._radius = value @property def area(self): import math return math.pi * self._radius
** 2
```

Class-based decorator (call counter):

```
class CallCounter: def __init__(self, func): self.func = func self.count = 0 def __call__(self,
*args, **kwargs): self.count += 1 print(f'Call #{self.count}') return self.func(*args, **kwargs)
@CallCounter def greet(name): print(f'Hello {name}') greet('Alice') # Call #1 greet('Bob') # Call #2
print(greet.count) # 2
```

■■ EXERCISES — Hands-on Practice

- ✓ Rewrite yesterday's timer/logger with `functools.wraps`
- ✓ Create a `Circle` class using `@property` for radius and area
- ✓ Build a class-based `rate_limiter` decorator (allows max N calls per minute)
- ✓ Verify `func.__name__` before and after applying `@wraps`
- ✓ Create `@classmethod` factory method for creating `Circle` from diameter

■ REAL-WORLD USE CASES

- `@property` used in SQLAlchemy ORM column definitions
- Django models use `@property` for computed fields
- `@classmethod` used in `datetime.datetime.fromisoformat()`
- Class-based decorators used in frameworks for middleware

■ Common Mistakes

- Forgetting `functools.wraps` — breaks introspection tools
- Using `@property` without understanding it breaks assignment
- Confusing `@classmethod (cls)` and `@staticmethod (no self/cls)`
- Thinking stacked decorators run top to bottom (they run bottom to top)

■ Best Practices

- Always use `@wraps` in production decorators
- Use `@property` for computed attributes, not simple getters
- Use `@classmethod` for alternative constructors
- Use `@staticmethod` for pure utility functions that don't need `self/cls`

■ INTERVIEW QUESTIONS

- Why do we use `functools.wraps`? What problem does it solve?
- What is the difference between `@classmethod` and `@staticmethod`?
- When would you use a class-based decorator vs a function decorator?
- What is `@property` and why is it better than `get/set` methods?
- If you stack `@a @b` on a function, which runs first?

■ DAILY LEETCODE — Easy

- #121 Best Time to Buy and Sell Stock — Easy
- #53 Maximum Subarray — Easy (Kadane's Algorithm)

■ GITHUB TASK

- Add `day2_decorators_advanced.py` | Update README with decorator cheatsheet table

■ NOTES TO CREATE

- Add `functools.wraps` explanation + before/after comparison to notes
- Create cheatsheet: `@property` vs `@classmethod` vs `@staticmethod`

■ TODAY'S DELIVERABLES

- ✓ File: `week1/day2_decorators_advanced.py`
- ✓ `functools.wraps` added to all yesterday's decorators
- ✓ Circle class with `@property`, `@setter`, `@classmethod`
- ✓ Class-based call counter decorator working
- ✓ GitHub commit pushed

DAY 3 Generators & Iterators — Lazy Evaluation

13 May 2026 (Wed) | ■ 3-4 hrs

■ THEORY — What to Learn

Imagine you need to process a 10GB log file. If you load it all into memory — crash. Generators solve this by producing values ONE AT A TIME, on demand. They are lazy — they don't compute anything until you ask for it. This is a critical concept for backend engineering, data pipelines, and streaming systems.

- Iterables vs Iterators vs Generators
 - Iterable: has `__iter__` method (list, dict, str)
 - Iterator: has both `__iter__` and `__next__`
 - Generator: special iterator using `yield` keyword
- `yield` keyword
 - Pauses function execution, saves state
 - Returns value to caller without ending function
 - Function resumes from `yield` on `next()` call

- Generator expressions
 - Like list comprehensions but lazy: `(x*2 for x in range(100))`
 - Uses `()` not `[]` — memory efficient
- yield from
 - Delegates to another generator/iterable
 - Cleaner syntax for nested generators
 - Used heavily in async Python
- Generator use cases
 - Large file reading line by line
 - Infinite sequences (ID generators)
 - Data pipeline processing
 - Memory-efficient data transformations

■ CODE — What to Write

Basic generator with yield:

```
def count_up(n): i = 0 while i < n: yield i # pause here, give value i += 1 g = count_up(3)
print(next(g)) # 0 print(next(g)) # 1 print(next(g)) # 2 # print(next(g)) # StopIteration!
```

Real-world: Read large file without loading into RAM:

```
def read_large_file(filepath): with open(filepath) as f: for line in f: yield line.strip() # Process
10GB file with ~1KB RAM: for line in read_large_file('huge.log'): process(line)
```

Infinite ID generator:

```
def id_generator(start=1): while True: yield start start += 1 gen = id_generator() print(next(gen)) #
1 print(next(gen)) # 2 # Never ends!
```

Generator expression vs list comprehension:

```
# List — loads ALL into memory nums_list = [x**2 for x in range(1_000_000)] # Generator — lazy,
produces one at a time nums_gen = (x**2 for x in range(1_000_000)) import sys
print(sys.getsizeof(nums_list)) # ~8MB print(sys.getsizeof(nums_gen)) # 112 bytes!
```

■■ EXERCISES — Hands-on Practice

- ✓ Build a CSV file reader generator (read row by row)
- ✓ Build a Fibonacci generator (infinite sequence)
- ✓ Compare memory: list comprehension vs generator expression for 1M items
- ✓ Build a pipeline: `read_file | filter_lines | parse_lines` using generators
- ✓ Build a custom range generator that accepts float steps

■ REAL-WORLD USE CASES

- Django QuerySets use lazy evaluation (like generators)
- FastAPI uses async generators for streaming responses
- pandas reads CSV chunks using generators
- Kafka/RabbitMQ consumers use generators for message processing

■■ Common Mistakes

- Trying to use `len()` on a generator — generators have no length
- Iterating a generator twice — second time gives nothing (exhausted)
- Using `list()` on infinite generator — hangs forever
- Forgetting generators are stateful — can't reset without recreating

■ Best Practices

- Use generators for large datasets, files, streams
- Use list comprehensions only when you need random access or `len()`
- Name generator functions clearly: `read_lines()`, `generate_ids()`
- Use `itertools` for complex generator combinations

■ INTERVIEW QUESTIONS

- What is the difference between yield and return?
- What is lazy evaluation and why does it matter?
- How would you read a 10GB file in Python without running out of memory?
- What is the difference between a generator expression and a list comprehension?
- What happens when a generator is exhausted?

■ DAILY LEETCODE — Easy

- #206 Reverse Linked List – Easy
- #141 Linked List Cycle – Easy

■ GITHUB TASK

- Add day3_generators.py | Add memory comparison script | Push

■ NOTES TO CREATE

- Diagram: Iterable → Iterator → Generator hierarchy
- Cheatsheet: when to use generator vs list

■ TODAY'S DELIVERABLES

- ✓ File: week1/day3_generators.py
- ✓ 5 generator functions working and tested
- ✓ Memory comparison script showing generator vs list sizes
- ✓ GitHub commit pushed
- ✓ Notes: generators vs iterators vs iterables diagram

DAY 4

Context Managers — The 'with' Statement Internals

14 May 2026 (Thu) | ■ 3-4 hrs

■ THEORY — What to Learn

You use 'with open(file) as f' every day. But do you know what happens internally? Context managers guarantee cleanup code runs — even if an exception occurs. This is critical in backend development: database connections, file handles, network sockets, and locks MUST be properly closed. Context managers ensure this.

- Protocol: `__enter__` and `__exit__`
 - `__enter__` runs when 'with' block starts
 - `__exit__` runs when block ends (even on exception)
 - `__exit__` receives exception type, value, traceback
- contextlib module
 - `@contextmanager` decorator — create context manager from generator
 - `contextlib.suppress` — silently ignore specific exceptions
 - `contextlib.ExitStack` — dynamic context managers
- Exception handling in context managers
 - If `__exit__` returns True — exception is suppressed
 - If `__exit__` returns False/None — exception propagates
 - This is powerful for custom error handling
- Async context managers
 - `__aenter__` and `__aexit__` for async/await contexts

- Used with 'async with' statement
- Common in database connections (asyncpg, aiomysql)

■ CODE — What to Write

Class-based context manager:

```
class DatabaseConnection:
    def __init__(self, url):
        self.url = url
        self.conn = None
    def __enter__(self):
        print(f'Connecting to {self.url}')
        self.conn = connect(self.url) # imagine this
        return self.conn
    def __exit__(self, exc_type, exc_val, exc_tb):
        print('Closing connection')
        self.conn.close()
        return False # don't suppress exceptions with
DatabaseConnection('postgresql://...') as db:
    db.execute('SELECT 1')
```

Generator-based with @contextmanager:

```
from contextlib import contextmanager
@contextmanager
def timer(label):
    import time
    start = time.time()
    try:
        yield # code inside 'with' block runs here
    finally:
        elapsed = time.time() - start
        print(f'{label}: {elapsed:.4f}s')
with timer('database query'):
    # run heavy operation
    time.sleep(0.5) # database query: 0.5001s
```

Suppress exceptions cleanly:

```
from contextlib import suppress
# Without suppress:
try:
    os.remove('file.txt')
except FileNotFoundError:
    pass
# With suppress (cleaner):
with suppress(FileNotFoundError):
    os.remove('file.txt')
```

■ EXERCISES — Hands-on Practice

- ✓ Build a ManagedFile context manager (open + close file safely)
- ✓ Build a Timer context manager using @contextmanager
- ✓ Build a TempDirectory context manager (creates + deletes temp dir)
- ✓ Build a Retry context manager that retries the with-block on exception
- ✓ Test that __exit__ runs even when exception occurs inside block

■ REAL-WORLD USE CASES

- SQLAlchemy uses context managers for transactions: with session.begin()
- Django test client uses context managers for request mocking
- Python threading.Lock() is a context manager
- aiohttp client sessions use async context managers

■ Common Mistakes

- Not handling exceptions in __exit__ — resources may still leak
- Returning True from __exit__ accidentally suppresses real errors
- Forgetting 'yield' in @contextmanager — nothing is given to 'as' variable
- Not using 'finally' in @contextmanager — cleanup may not run

■ Best Practices

- Always use context managers for file I/O, DB connections, locks
- Use @contextmanager for simple cases, class for complex state
- Use contextlib.suppress instead of empty try/except
- Return the resource from __enter__ to make 'as' useful

■ INTERVIEW QUESTIONS

- What is a context manager and what problem does it solve?
- What methods does a class need to implement to be a context manager?
- What happens if an exception occurs inside a 'with' block?
- What is the difference between class-based and @contextmanager approach?
- Name 3 real-world uses of context managers in backend development.

■ DAILY LEETCODE — Easy

→ #104 Maximum Depth of Binary Tree – Easy

→ #226 Invert Binary Tree – Easy

■ GITHUB TASK

- Add day4_context_managers.py | Test exception safety | Push

■ NOTES TO CREATE

- `__enter__` / `__exit__` protocol cheatsheet
- `@contextmanager` template to reuse in future projects

■ TODAY'S DELIVERABLES

- ✓ File: week1/day4_context_managers.py
- ✓ 4 context managers built and tested
- ✓ Exception handling tested (verify cleanup runs on crash)
- ✓ GitHub pushed
- ✓ Notes: context manager protocol diagram

DAY 5

Async/Await & asyncio — Concurrent Python

15 May 2026 (Fri) | ■ 3-4 hrs

■ THEORY — What to Learn

Async is NOT about running code faster on one task. It is about doing MANY things at the same time when you are WAITING. Example: fetching 100 URLs. Synchronously = wait for URL 1, then URL 2, then URL 3... Async = send ALL 100 requests simultaneously, collect results as they arrive. This is why FastAPI is so fast — it is fully async.

- Concurrency vs Parallelism vs Async
 - Parallelism: multiple CPU cores doing work simultaneously
 - Concurrency: single thread switching between tasks while waiting
 - Async: concurrency for I/O-bound tasks (network, file, DB)
- Event Loop
 - The engine that runs all async code
 - Continuously checks: is any task ready to continue?
 - `asyncio.run()` creates and runs the event loop
- `async def` and `await`
 - `async def`: marks a function as a coroutine
 - `await`: pauses this coroutine, lets event loop run others
 - Can only use `await` inside `async def`
- `asyncio.gather()` and `asyncio.create_task()`
 - `gather()`: run multiple coroutines concurrently, wait for all
 - `create_task()`: schedule coroutine without waiting immediately
 - Key difference: `gather()` blocks until all done
- `aiohttp` for async HTTP
 - Replacement for `requests` library in async code
 - Use with 'async with `aiohttp.ClientSession()`' pattern

■ CODE — What to Write

Basic async function:

```
import asyncio
async def fetch_data(url):
    await asyncio.sleep(1) # simulates network wait
    return f'Data from {url}'
async def main():
    result = await fetch_data('api.example.com')
    print(result)
asyncio.run(main())
```

asyncio.gather — concurrent execution:

```
import asyncio
import time
async def fetch(n):
    await asyncio.sleep(1) # each takes 1 second
    return f'Result {n}'
async def main():
    start = time.time()
    # Without gather: 3 seconds total
    # With gather: ~1 second total!
    results = await asyncio.gather(fetch(1), fetch(2), fetch(3))
    print(f'Done in {time.time()-start:.2f}s')
    print(results)
asyncio.run(main()) # Done in 1.00s
```

Real aiohttp example:

```
import aiohttp
import asyncio
async def fetch_url(session, url):
    async with session.get(url) as response:
        return await response.json()
async def fetch_all(urls):
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_url(session, url) for url in urls]
        return await asyncio.gather(*tasks)
urls = ['https://api.github.com/users/user1', 'https://api.github.com/users/user2']
results = asyncio.run(fetch_all(urls))
```

■ EXERCISES — Hands-on Practice

- ✓ Build sync vs async comparison: fetch 5 URLs, measure time difference
- ✓ Build async task queue: process 20 items, max 5 concurrent
- ✓ Build async file reader using aiofiles library
- ✓ Create asyncio.create_task() example showing non-blocking execution
- ✓ Build a rate-limited async fetcher (max 10 requests/second)

■ REAL-WORLD USE CASES

- FastAPI is fully async — handles thousands of concurrent requests
- Discord bot libraries use asyncio event loop
- Database drivers: asyncpg (PostgreSQL), motor (MongoDB) are async
- Web scrapers use aiohttp + asyncio to scrape 100s of pages simultaneously

■ Common Mistakes

- Using requests instead of aiohttp inside async functions
- Calling async function without await — returns coroutine object, not result
- Using time.sleep() instead of asyncio.sleep() — blocks event loop
- Mixing sync and async code without proper bridges
- Creating new event loop in async context instead of using running one

■ Best Practices

- Use async for I/O-bound work (HTTP, DB, files)
- Use multiprocessing for CPU-bound work (ML, image processing)
- Always use asyncio.run() at the entry point
- Use asyncio.gather() for parallel I/O operations
- Install aiofiles for async file operations

■ INTERVIEW QUESTIONS

- What is the difference between async and threading in Python?
- When should you use async vs multiprocessing?
- What is an event loop?
- What is the difference between asyncio.gather and asyncio.create_task?
- Why can't you use 'requests' library in async code?
- What is a coroutine?

■ DAILY LEETCODE — Easy

- #217 Contains Duplicate — Easy
- #242 Valid Anagram — Easy

■ GITHUB TASK

- Add day5_async.py | Create benchmark README table | Push

■ NOTES TO CREATE

- Async mental model diagram: event loop + coroutines
- When to use: async vs threading vs multiprocessing — decision table

■ TODAY'S DELIVERABLES

- ✓ File: week1/day5_async.py
- ✓ Sync vs async benchmark showing speedup
- ✓ Async URL fetcher using aiohttp + asyncio.gather
- ✓ GitHub pushed with benchmark results in README
- ✓ Notes: event loop diagram + async mental model

DAY 6

SATURDAY — Build: Async CLI Tool + Week Review

16 May 2026 (Sat) | ■ 4 hrs

■ THEORY — What to Learn

Saturday is for BUILDING. You've learned decorators, generators, context managers, and async. Today you combine them all into one real project: an Async Stock Price Fetcher CLI tool. This simulates a real backend task you would do at a company.

- Project: Async Stock Price Fetcher
 - Fetch prices for multiple stocks simultaneously using aiohttp
 - Use decorators for timing and logging
 - Use generators to yield stock symbols from CSV
 - Use context managers for file output
- Folder structure to create
 - stock_fetcher/
 - main.py — entry point with asyncio.run()
 - fetcher.py — async fetch functions
 - decorators.py — timer + logger
 - utils.py — file reading generator
 - README.md — usage instructions

■ CODE — What to Write

Project entry point:

```
# main.py import asyncio from fetcher import fetch_all_stocks from decorators import timer from utils
import read_symbols @timer async def main(): symbols = list(read_symbols('stocks.csv')) prices =
await fetch_all_stocks(symbols) for symbol, price in prices.items(): print(f'{symbol}: ${price}')
asyncio.run(main())
```

fetcher.py:

```
import aiohttp import asyncio ASYNC def fetch_price(session, symbol): url =
f'https://query1.finance.yahoo.com/v8/finance/chart/{symbol}' async with session.get(url) as resp:
data = await resp.json() return symbol, data['chart']['result'][0]['meta']['regularMarketPrice']
async def fetch_all_stocks(symbols): async with aiohttp.ClientSession() as session: tasks =
[fetch_price(session, s) for s in symbols] results = await asyncio.gather(*tasks,
return_exceptions=True) return {s: p for s, p in results if not isinstance(p, Exception)}
```

■■ EXERCISES — Hands-on Practice

- ✓ Create all project files with proper structure

- ✓ Implement `read_symbols` generator for CSV reading
- ✓ Implement async fetch with error handling
- ✓ Add timing decorator showing total fetch time
- ✓ Add file output context manager saving results to JSON
- ✓ Test with 10 stock symbols simultaneously
- ✓ Write README.md with usage instructions

■ REAL-WORLD USE CASES

- This pattern is used in fintech apps for real-time price aggregation
- Similar to how trading platforms fetch data from multiple exchanges

■■ Common Mistakes

- No error handling for failed requests
- Hardcoding symbols instead of reading from file
- Not using context manager for aiohttp session

■ Best Practices

- Separate concerns: `fetcher.py`, `decorators.py`, `utils.py`
- Use `return_exceptions=True` in `gather` to handle partial failures
- Always close aiohttp session (use `async with`)

■ INTERVIEW QUESTIONS

- Walk me through your async stock fetcher — why did you design it this way?
- How would you add rate limiting to avoid hitting API limits?
- How would you handle 1000 stocks efficiently?

■ DAILY LEETCODE — Easy

- #1 Two Sum – revisit, write optimal $O(n)$ solution
- #125 Valid Palindrome – Easy

■ GITHUB TASK

- Push complete `stock_fetcher` project | Add README with demo output

■ NOTES TO CREATE

- Project architecture diagram in notes/
- Lessons learned from building this project

■ TODAY'S DELIVERABLES

- ✓ Complete project folder: `stock_fetcher/`
- ✓ All 5 Python files implemented and working
- ✓ README.md with setup instructions
- ✓ GitHub repo created and pushed
- ✓ Screenshot of CLI output in README

DAY 7

SUNDAY — OOP Deep Dive Preview + Week 1 Revision

17 May 2026 (Sun) | ■ 4 hrs

■ THEORY — What to Learn

Sunday = Review + Prepare. Spend the first 2 hours reviewing this week's content. Then spend 2 hours previewing next week's topic: Advanced OOP. This overlap creates a memory bridge — you won't start Monday cold.

- Revision: This week's topics
 - Decorators: basic, advanced, `functools.wraps`, stacking

- Generators: yield, generator expressions, yield from
- Context managers: `__enter__`/`__exit__`, `@contextmanager`
- Async/Await: event loop, `asyncio.gather`, `aiohttp`
- Preview: Next week — Advanced OOP
- Dunder/magic methods: `__str__`, `__repr__`, `__len__`, `__eq__`
- Abstract Base Classes (ABC)
- Dataclasses — modern Python OOP
- Design Patterns: Singleton, Factory, Observer

■ CODE — What to Write

Quick revision quiz — answer without looking:

```
# Q1: What does this print? import functools def dec(func): def wrapper(*args): return func(*args)
return wrapper @dec def hello(): pass print(hello.__name__) # 'wrapper' (forgot @wraps!) # Q2: How
many items in memory? gen = (x**2 for x in range(1_000_000)) # Answer: 1 item at a time! # Q3: Is
this valid? async def fetch(): return requests.get('http://api.com') # NO! Use aiohttp
```

■■ EXERCISES — Hands-on Practice

- ✓ Redo the timer decorator from memory (no looking)
- ✓ Write a generator that produces squares of even numbers up to N
- ✓ Write a context manager using `@contextmanager` that logs entry/exit
- ✓ Write an async function that fetches 3 URLs and returns all results
- ✓ Solve 3 LeetCode Easy problems

■■ Common Mistakes

- Skipping Sunday revision — worst habit for long-term retention
- Not writing notes in your own words — copying doesn't help you remember

■ Best Practices

- Use Feynman technique: explain each concept out loud
- Write week summary in your own words — not copy-paste
- Preview next week on Sunday — reduces Monday morning difficulty

■ INTERVIEW QUESTIONS

- Explain decorators to a 10-year-old.
- What is the difference between a generator and a list?
- Why is async Python important for backend development?

■ DAILY LEETCODE — Easy

- #21 Merge Two Sorted Lists — Easy
- #83 Remove Duplicates from Sorted List — Easy
- #206 Reverse Linked List — revisit for speed

■ GITHUB TASK

- Create `notes/week1/week1_summary.md` | Push final week state

■ NOTES TO CREATE

- `week1_summary.md` — cover all 5 topics in your own words
- Interview Q&A; document — questions + your answers

■ TODAY'S DELIVERABLES

- ✓ All 4 revision exercises completed without looking at notes
- ✓ `notes/week1/week1_summary.md` created — your own words
- ✓ 3 LeetCode Easy problems solved

- ✓ GitHub up to date with all week's work
- ✓ Next week topics previewed and noted

■ TRADING — Personal Learning | Week 1

■ This is SEPARATE from your daily 3-4hr tech learning. Do this in evenings (30-45 min).

- What is the stock market and how does it work?
- Difference between NSE and BSE (Indian exchanges)
- What are indices: Nifty 50, Sensex — how they are calculated
- Types of stocks: large-cap, mid-cap, small-cap
- How to open a Zerodha / Groww / Upstox account (paper trading first)
- Understanding a stock quote: Open, High, Low, Close, Volume
- Difference between equity and derivatives
- Resource: Watch 'Stock Market for Beginners' by Zerodha Varsity (free)

■ REVISION WEEK 1 | After Week 2 (25 May 2026)

■ Topics to Revise

- Python Decorators — all types, functools.wraps, stacking
- Generators — yield, generator expressions, yield from, memory efficiency
- Context Managers — `__enter__`/`__exit__`, `@contextmanager`, `contextlib`
- Async/Await — event loop, `asyncio.gather`, `aiohttp`
- OOP Magic Methods — `__str__`, `__repr__`, `__len__`, `__eq__`
- Async Stock Fetcher project — explain architecture

■ Revision Activities

- Re-read your notes from this period
- Redo 2-3 coding exercises without looking at solutions
- Explain each concept out loud (Feynman Technique)
- Update GitHub README for any projects built
- Solve 5 LeetCode problems on these topics
- Create a mind-map of concepts learned
- Watch 1 review video per major topic
- Write a blog-style summary (even if private)

WEEK 2 — Advanced OOP + Design Patterns | 18–24 May 2026

DAY 8 Magic Methods / Dunder Methods

18 May 2026 (Mon) | ■ 3-4 hrs

■ THEORY — What to Learn

Magic methods (also called dunder methods — double underscore) are special methods Python calls automatically in certain situations. When you write `len(my_list)`, Python calls `my_list.__len__()`. When you write `a + b`, Python calls `a.__add__(b)`. Understanding these lets you create objects that behave like built-in Python types — a key skill for

writing Pythonic code and passing senior interviews.

- String representation: `__str__` vs `__repr__`
 - `__str__`: human-readable, used by `print()` and `str()`
 - `__repr__`: developer-friendly, used in REPL and debugging
 - Rule: `__repr__` should return string that recreates object
- Comparison methods
 - `__eq__`: `==` operator
 - `__lt__`: `<` operator (used by `sorted()` and `min/max`)
 - `__le__`, `__gt__`, `__ge__`: `<=`, `>`, `>=`
 - `@functools.total_ordering`: define `__eq__` + one, get all
- Math operators
 - `__add__`: `+` operator
 - `__mul__`: `*` operator
 - `__sub__`, `__truediv__`, `__floordiv__`, `__mod__`
- Container methods
 - `__len__`: `len()` function
 - `__getitem__`: `obj[key]` access
 - `__setitem__`: `obj[key] = value`
 - `__contains__`: `'in'` operator
 - `__iter__`: for `x in obj` iteration
- Lifecycle methods
 - `__init__`: constructor
 - `__del__`: destructor (avoid in production)
 - `__new__`: object creation (Singleton pattern)

■ CODE — What to Write

Complete custom Vector class:

```
from functools import total_ordering @total_ordering class Vector: def __init__(self, x, y): self.x = x self.y = y def __repr__(self): return f'Vector({self.x}, {self.y})' def __str__(self): return f'({self.x}, {self.y})' def __add__(self, other): return Vector(self.x + other.x, self.y + other.y) def __mul__(self, scalar): return Vector(self.x * scalar, self.y * scalar) def __len__(self): return int((self.x**2 + self.y**2) ** 0.5) def __eq__(self, other): return (self.x, self.y) == (other.x, other.y) def __lt__(self, other): return len(self) < len(other) v1 = Vector(3, 4) v2 = Vector(1, 2) print(v1 + v2) # (4, 6) print(v1 * 3) # (9, 12) print(len(v1)) # 5 print(v1 > v2) # True (from @total_ordering)
```

■■ EXERCISES — Hands-on Practice

- ✓ Build a Money class with `+`, `-`, comparison and `str` representation
- ✓ Build a Stack class with `__len__`, `__contains__`, `__iter__`
- ✓ Build a Matrix class with `+` and `*` operators
- ✓ Use `@total_ordering` to add all comparisons from just `__eq__` + `__lt__`
- ✓ Test all magic methods interactively in Python REPL

■ REAL-WORLD USE CASES

- Django models use `__str__` to display objects in admin panel
- NumPy arrays use `__add__`, `__mul__` for vectorized math
- Pandas DataFrames use `__getitem__` for `df['column']` syntax
- SQLAlchemy uses `__eq__` for query filter conditions

■ Common Mistakes

- Implementing `__eq__` without `__hash__` — breaks dict/set usage
- Returning string from `__repr__` that doesn't recreate the object
- Using `__del__` for cleanup — use context managers instead
- Forgetting to return `NotImplemented` from operator methods on type mismatch

■ Best Practices

- Always implement both `__str__` and `__repr__`
- Use `@total_ordering` to avoid implementing all 6 comparison methods
- If you define `__eq__`, also define `__hash__` (or set to `None`)
- Make `__repr__` return valid Python that recreates the object

■ INTERVIEW QUESTIONS

- What is the difference between `__str__` and `__repr__`?
- How does Python know how to handle the `+` operator on custom objects?
- What magic method does the `'in'` operator use?
- How would you make a custom class sortable?
- What is `@functools.total_ordering`?

■ DAILY LEETCODE — Easy

- #232 Implement Queue using Stacks — Easy
- #225 Implement Stack using Queues — Easy

■ GITHUB TASK

- Create `week2/` folder | Add `day8_magic_methods.py` | Push

■ NOTES TO CREATE

- Magic methods reference table: Method → Triggered by
- Vector class as template for operator overloading

■ TODAY'S DELIVERABLES

- ✓ File: `week2/day8_magic_methods.py`
- ✓ Vector, Money, Stack classes with all magic methods
- ✓ Tests showing all operators work correctly
- ✓ GitHub pushed

DAY 9 Abstract Base Classes + Dataclasses

19 May 2026 (Tue) | ■ 3-4 hrs

■ THEORY — What to Learn

Abstract Base Classes (ABC) enforce a contract: any class that inherits from your ABC MUST implement certain methods, or Python raises an error. This is Python's way of doing interfaces. Dataclasses are the modern (Python 3.7+) way to write simple data-holding classes — they auto-generate `__init__`, `__repr__`, `__eq__` for you.

- Abstract Base Classes (abc module)
 - `from abc import ABC, abstractmethod`
 - `@abstractmethod`: marks method as required in subclass
 - Cannot instantiate an ABC directly
 - Used to define interfaces/contracts in Python
- Dataclasses (`@dataclass`)
 - Auto-generates: `__init__`, `__repr__`, `__eq__`
 - `from dataclasses import dataclass, field`
 - `field()` for default values and metadata

- frozen=True makes dataclass immutable (like a named tuple)
- dataclass vs NamedTuple vs TypedDict
 - dataclass: mutable, OOP-friendly, most flexible
 - NamedTuple: immutable, tuple-like, hashable
 - TypedDict: for dict typing hints, not a class
- Post-init processing
 - __post_init__ method called after auto-generated __init__
 - Used for validation and derived fields

■ CODE — What to Write

Abstract Base Class — payment systems:

```
from abc import ABC, abstractmethod
class PaymentProcessor(ABC):
    @abstractmethod
    def process_payment(self, amount: float) -> bool: pass
    @abstractmethod
    def refund(self, transaction_id: str) -> bool: pass
    def validate_amount(self, amount): # Concrete method – shared by all subclasses
    return amount > 0
class StripeProcessor(PaymentProcessor):
    def process_payment(self, amount):
    print(f'Processing ${amount} via Stripe')
    return True
    def refund(self, transaction_id):
    print(f'Refunding {transaction_id} via Stripe')
    return True
# p = PaymentProcessor() # ERROR: Can't instantiate ABC
p = StripeProcessor() # OK
```

Dataclass example:

```
from dataclasses import dataclass, field
from datetime import datetime
@dataclass
class User:
    name: str
    email: str
    age: int
    created_at: datetime = field(default_factory=datetime.now)
    tags: list = field(default_factory=list)
    def __post_init__(self):
    if self.age < 0:
    raise ValueError('Age cannot be negative')
    self.email = self.email.lower()
u = User('Alice', 'Alice@EMAIL.com', 25)
print(u) # User(name='Alice', email='alice@email.com', ...)
print(u == User('Alice', 'alice@email.com', 25)) # True (auto __eq__)
```

■■ EXERCISES — Hands-on Practice

- ✓ Build an Animal ABC with speak() and move() abstract methods
- ✓ Implement Dog, Cat, Bird classes from Animal ABC
- ✓ Try instantiating the ABC directly — observe error
- ✓ Create a Product dataclass with name, price, quantity
- ✓ Add __post_init__ validation to Product (price > 0)
- ✓ Make a frozen Product dataclass and try mutating it

■ REAL-WORLD USE CASES

- FastAPI uses ABC for dependency injection interfaces
- Django has abstract model base classes for shared fields
- Python's io module is built entirely on ABCs
- pydantic BaseModel is similar to but more powerful than dataclass

■■ Common Mistakes

- Forgetting to inherit from ABC in abstract class
- Implementing all abstract methods but forgetting one — still can't instantiate
- Using mutable default values in dataclass without field(default_factory=...)
- Expecting dataclass to generate __hash__ when eq=True (it sets __hash__=None)

■ Best Practices

- Use ABC to enforce interfaces in large codebases
- Use dataclass for simple data containers instead of plain classes
- Use frozen=True for configuration objects that shouldn't change
- Always validate in __post_init__ for data integrity

■ INTERVIEW QUESTIONS

- What is an Abstract Base Class and why would you use it?
- What does @abstractmethod do?

- What methods does `@dataclass` auto-generate?
- When would you use a frozen dataclass?
- What is `__post_init__` used for?

■ DAILY LEETCODE — Medium

- #155 Min Stack — Medium
- #20 Valid Parentheses — revisit faster

■ GITHUB TASK

- Add `day9_abc_dataclasses.py` | Push

■ NOTES TO CREATE

- ABC vs Interface comparison table
- Dataclass vs dict vs NamedTuple decision guide

■ TODAY'S DELIVERABLES

- ✓ File: `week2/day9_abc_dataclasses.py`
- ✓ ABC + 3 concrete implementations working
- ✓ Dataclass with validation and field defaults
- ✓ GitHub pushed

DAY 10

Design Patterns — Singleton, Factory, Observer

20 May 2026 (Wed) | ■ 3-4 hrs

■ THEORY — What to Learn

Design patterns are proven solutions to common programming problems. They are not code — they are blueprints. In interviews, knowing design patterns shows you think like a senior engineer. In real projects, they make your code maintainable and extensible. Today: 3 most important for backend development.

- Singleton Pattern
 - Ensures only ONE instance of a class exists
 - Used for: database connections, config managers, loggers
 - Implementation: using `__new__` or module-level variable
 - Warning: harder to test — use dependency injection instead in production
- Factory Pattern
 - Creates objects without specifying exact class
 - Caller says WHAT they want, factory decides HOW to create it
 - Types: Factory Method, Abstract Factory
 - Used for: payment processors, notification services, DB drivers
- Observer Pattern
 - One object (Subject) notifies many (Observers) of state changes
 - Loose coupling — subject doesn't know who is observing
 - Used for: event systems, GUI, webhooks, pub/sub messaging
 - Python: can use signals, or implement manually
- Strategy Pattern (bonus)
 - Defines family of algorithms, encapsulates each, makes them interchangeable
 - Used for: sorting algorithms, payment methods, compression

■ CODE — What to Write

Singleton — Database Connection Manager:

```
class DatabaseManager:
    __instance = None
    def __new__(cls):
        if cls.__instance is None:
            cls.__instance = super().__new__(cls)
            cls.__instance._connected = False
        return cls.__instance
    def connect(self, url):
        if not self._connected:
            print(f'Connecting to {url}')
            self._connected = True
        db1 = DatabaseManager()
        db2 = DatabaseManager()
        print(db1 is db2) # True - same instance!
```

Factory — Payment Processor:

```
class PaymentFactory:
    @staticmethod
    def create(payment_type: str):
        processors = {
            'stripe': StripeProcessor,
            'razorpay': RazorpayProcessor,
            'paytm': PaytmProcessor,
        }
        processor = processors.get(payment_type)
        if not processor:
            raise ValueError(f'Unknown: {payment_type}')
        return processor()
    # Usage: processor = PaymentFactory.create('razorpay')
    processor.process_payment(1000)
```

Observer — Stock Price Alert:

```
class StockMarket:
    def __init__(self):
        self._observers = []
        self._price = 0
    def subscribe(self, observer):
        self._observers.append(observer)
    def notify(self):
        for obs in self._observers:
            obs.update(self._price)
    def set_price(self, price):
        self._price = price
        self.notify()
    class PriceAlert:
        def __init__(self, threshold):
            self.threshold = threshold
        def update(self, price):
            if price > self.threshold:
                print(f'ALERT: Price {price} exceeded {self.threshold}!')
    market = StockMarket()
    market.subscribe(PriceAlert(1000))
    market.subscribe(PriceAlert(1500))
    market.set_price(1200) # ALERT: 1200 exceeded 1000!
```

■■ EXERCISES — Hands-on Practice

- ✓ Implement Singleton logger that writes to file
- ✓ Build Factory for notification types: Email, SMS, Push
- ✓ Build Observer for a simple event system
- ✓ Combine all 3: Singleton factory that notifies observers
- ✓ Find 2 examples of each pattern in Python standard library

■ REAL-WORLD USE CASES

- Django signals use Observer pattern
- SQLAlchemy session uses Factory pattern
- Python logging module uses Singleton-like pattern
- celery uses Observer for task status callbacks

■■ Common Mistakes

- Singleton in multi-threaded apps — not thread-safe by default
- Overusing Singleton — creates hidden global state
- Factory without proper error handling for unknown types
- Observer with memory leaks — observers not unsubscribed

■ Best Practices

- Prefer dependency injection over Singleton in production
- Factory pattern is great for plugin/adaptor architectures
- Observer should have unsubscribe capability
- Name patterns explicitly in code: PaymentFactory, not just Payment

■ INTERVIEW QUESTIONS

- What is the Singleton pattern and when would you use it?
- Explain the Factory pattern with a real-world example.
- What is the Observer pattern? Where is it used in web frameworks?
- What are the downsides of the Singleton pattern?
- What design pattern does Django signals implement?

■ DAILY LEETCODE — Medium

- #3 Longest Substring Without Repeating Characters — Medium
- #167 Two Sum II — Medium

■ GITHUB TASK

- Add day10_design_patterns.py | Push with pattern explanations in comments

■ NOTES TO CREATE

- Design patterns cheatsheet: Pattern → Problem → Solution → Example
- When to use which pattern — decision flowchart

■ TODAY'S DELIVERABLES

- ✓ File: week2/day10_design_patterns.py
- ✓ All 3 patterns implemented and tested
- ✓ README explaining each pattern in plain English
- ✓ GitHub pushed

DAY 11

Type Hints & mypy — Static Typing in Python

21 May 2026 (Thu) | ■ 3-4 hrs

■ THEORY — What to Learn

Python is dynamically typed but you CAN add type hints for safety. mypy checks these at development time. FastAPI and Pydantic require good type hints. This is now standard in all professional Python codebases.

- Core concepts for Type Hints & mypy — Static Typing in Python
- Practical implementation
- Common patterns and anti-patterns

■ EXERCISES — Hands-on Practice

- ✓ Complete main exercise for Type Hints & mypy — Static Typing in Python
- ✓ Write tests for the code you build
- ✓ Push to GitHub

■ REAL-WORLD USE CASES

- Used extensively in FastAPI projects
- Standard in all Python backends

■ Common Mistakes

- Skipping tests
- Not committing to GitHub daily

■ Best Practices

- Write tests first (TDD)
- Clear variable names

■ INTERVIEW QUESTIONS

- Explain Type Hints & mypy in 2 minutes
- What are the key benefits?
- When would you NOT use this?

■ DAILY LEETCODE — Medium

- #15 3Sum — Medium
- #11 Container With Most Water — Medium

■ GITHUB TASK

- Push day11 work with tests

■ NOTES TO CREATE

- Update running notes file with today's learnings

■ TODAY'S DELIVERABLES

- ✓ day11_type.py created
- ✓ Tests passing
- ✓ GitHub pushed
- ✓ Notes updated

DAY 12 Testing with pytest — TDD Basics

22 May 2026 (Fri) | ■ 3-4 hrs

■ THEORY — What to Learn

Tests are not optional in professional development. pytest is the industry standard. TDD (Test-Driven Development) = write test first, then code. Every function you write from now on should have a test.

- Core concepts for Testing with pytest — TDD Basics
- Practical implementation
- Common patterns and anti-patterns

■■ EXERCISES — Hands-on Practice

- ✓ Complete main exercise for Testing with pytest — TDD Basics
- ✓ Write tests for the code you build
- ✓ Push to GitHub

■ REAL-WORLD USE CASES

- Used extensively in FastAPI projects
- Standard in all Python backends

■■ Common Mistakes

- Skipping tests
- Not committing to GitHub daily

■ Best Practices

- Write tests first (TDD)
- Clear variable names

■ INTERVIEW QUESTIONS

- Explain Testing with pytest in 2 minutes
- What are the key benefits?
- When would you NOT use this?

■ DAILY LEETCODE — Medium

- #15 3Sum — Medium
- #11 Container With Most Water — Medium

■ GITHUB TASK

- Push day12 work with tests

■ NOTES TO CREATE

- Update running notes file with today's learnings

■ TODAY'S DELIVERABLES

- ✓ day12_testing.py created

- ✓ Tests passing
- ✓ GitHub pushed
- ✓ Notes updated

DAY 13 Saturday — Build: Tested OOP Library

23 May 2026 (Sat) | ■ 4 hrs

■ THEORY — What to Learn

Build a small Python library (e.g., a simple task management system) that uses: ABCs, dataclasses, design patterns, type hints, and full pytest test suite. This becomes a portfolio piece.

- Core concepts for Saturday — Build: Tested OOP Library
- Practical implementation
- Common patterns and anti-patterns

■ EXERCISES — Hands-on Practice

- ✓ Complete main exercise for Saturday — Build: Tested OOP Library
- ✓ Write tests for the code you build
- ✓ Push to GitHub

■ REAL-WORLD USE CASES

- Used extensively in FastAPI projects
- Standard in all Python backends

■ Common Mistakes

- Skipping tests
- Not committing to GitHub daily

■ Best Practices

- Write tests first (TDD)
- Clear variable names

■ INTERVIEW QUESTIONS

- Explain Saturday in 2 minutes
- What are the key benefits?
- When would you NOT use this?

■ DAILY LEETCODE — Medium

- #15 3Sum — Medium
- #11 Container With Most Water — Medium

■ GITHUB TASK

- Push day13 work with tests

■ NOTES TO CREATE

- Update running notes file with today's learnings

■ TODAY'S DELIVERABLES

- ✓ day13_saturday.py created
- ✓ Tests passing
- ✓ GitHub pushed
- ✓ Notes updated

THEORY — What to Learn

Revise all Week 2 topics. Preview Java fundamentals for next week. Solve 3 LeetCode Medium problems.

- Core concepts for Sunday — Revision Week 2 + Preview Month 2
- Practical implementation
- Common patterns and anti-patterns

EXERCISES — Hands-on Practice

- ✓ Complete main exercise for Sunday — Revision Week 2 + Preview Month 2
- ✓ Write tests for the code you build
- ✓ Push to GitHub

REAL-WORLD USE CASES

- Used extensively in FastAPI projects
- Standard in all Python backends

Common Mistakes

- Skipping tests
- Not committing to GitHub daily

Best Practices

- Write tests first (TDD)
- Clear variable names

INTERVIEW QUESTIONS

- Explain Sunday in 2 minutes
- What are the key benefits?
- When would you NOT use this?

DAILY LEETCODE — Medium

- #15 3Sum — Medium
- #11 Container With Most Water — Medium

GITHUB TASK

- Push day14 work with tests

NOTES TO CREATE

- Update running notes file with today's learnings

TODAY'S DELIVERABLES

- ✓ day14_sunday.py created
- ✓ Tests passing
- ✓ GitHub pushed
- ✓ Notes updated

■ MONTH 2 — Java + Docker | 11 Jun – 10 Jul 2026

Month 2 introduces Java — the backbone of enterprise backend development. Since you know Python, Java's syntax will feel familiar but stricter. You also start Docker — containerising your apps for cloud deployment.

Week	Topics	Project
Week 3 (11-17 Jun)	Java syntax, OOP, Collections	Bank Account System
Week 4 (18-24 Jun)	Streams, Lambdas, Generics	Data Processing CLI
Week 5 (25 Jun-1 Jul)	Exception handling, JUnit 5	Tested Library
Week 6 (2-8 Jul)	Docker: containers, Dockerfile, Compose	Dockerized App
Revision (9-10 Jul)	Java + Docker review	Architecture docs

DAY 22

Java OOP — Classes, Inheritance, Interfaces, Polymorphism

11 Jun 2026 (Thu) | ■ 3-4 hrs

■ THEORY — What to Learn

Java's OOP is stricter than Python's. Everything is a class. Java uses interfaces (like Python's ABCs) for contracts, and abstract classes for partial implementation. Polymorphism lets you write code that works with multiple types. The key insight: Java forces you to think about types — which makes you a better programmer.

- Classes and Objects
 - public class vs package-private
 - Constructor overloading
 - this keyword
- Inheritance
 - extends keyword
 - super() call
 - Method overriding with @Override
- Interfaces vs Abstract Classes
 - Interface: contract only, no state (Java 8+: default methods)
 - Abstract class: partial implementation, has state
 - Class can implement MULTIPLE interfaces but extend ONE class
- Polymorphism
 - Same method name, different behavior depending on object type
 - Runtime polymorphism via method overriding
 - Compile-time polymorphism via method overloading

■ CODE — What to Write

Java class hierarchy example:

```
// Interface public interface Payable { boolean processPayment(double amount); default void logPayment(double amount) { System.out.println("Payment: " + amount); } } // Abstract class public abstract class BaseProcessor implements Payable { protected String name; public BaseProcessor(String name) { this.name = name; } // Concrete shared method public boolean validateAmount(double amount) { return amount > 0; } } // Concrete class public class StripeProcessor extends BaseProcessor { public StripeProcessor() { super("Stripe"); } @Override public boolean processPayment(double amount) { if (!validateAmount(amount)) return false; logPayment(amount); return true; } }
```

■■ EXERCISES — Hands-on Practice

- ✓ Create Animal hierarchy: Animal (abstract) → Dog, Cat, Bird
- ✓ Build Shape interface with area() and perimeter()
- ✓ Implement Circle, Rectangle, Triangle from Shape
- ✓ Demonstrate polymorphism: List with mixed types
- ✓ Build BankAccount with Checking and Savings subclasses

■ REAL-WORLD USE CASES

- Spring Boot extensively uses interface injection
- Java Collections Framework is built on interfaces (List, Map, Set)
- JDBC uses interfaces for database drivers

■■ Common Mistakes

- Forgetting @Override — method doesn't actually override, creates new method
- Trying to instantiate an abstract class
- Confusing is-a (inheritance) with has-a (composition) relationships

■ Best Practices

- Prefer interfaces over abstract classes (composition over inheritance)
- Always use @Override annotation for clarity
- Program to interface, not implementation
- Keep class hierarchy shallow (max 2-3 levels)

■ INTERVIEW QUESTIONS

- What is the difference between an interface and an abstract class in Java?
- Can a class implement multiple interfaces? Can it extend multiple classes?
- What is the difference between method overriding and method overloading?
- What does @Override annotation do?
- Why is polymorphism important?

■ DAILY LEETCODE — Medium

- #200 Number of Islands — Medium
- #133 Clone Graph — Medium

■ GITHUB TASK

- Create java-learning/ repo | Push OOP examples | Add Maven pom.xml

■ NOTES TO CREATE

- Java vs Python OOP comparison table
- Interface vs Abstract Class decision guide

■ TODAY'S DELIVERABLES

- ✓ Java project structure in IntelliJ IDEA
- ✓ Animal + Shape hierarchies implemented
- ✓ BankAccount system with polymorphism
- ✓ GitHub pushed (java-learning repo)

■ MONTH 3 — Spring Boot Intermediate | 11 Jul – 10 Aug 2026

Spring Boot is the most popular Java backend framework in the world. Used by thousands of companies including Amazon, Netflix, LinkedIn, and most Indian MNCs. By end of this month, you will be able to build production-ready REST APIs with authentication, database integration, and tests.

Week	Topics	Project
Week 7 (11-17 Jul)	Spring Boot setup, REST API, MVC, Annotations	Todo REST API
Week 8 (18-24 Jul)	JPA/Hibernate, PostgreSQL, Relationships	Blog API with DB
Week 9 (25-31 Jul)	Spring Security, JWT Auth, Exception Handling	Auth API
Week 10 (1-7 Aug)	Testing: JUnit5, Mockito, Integration Tests	Fully Tested API
Revision (8-10 Aug)	Spring Boot full review	API Docs + Postman

■ MILESTONE: PHASE 1 COMPLETE — Python Advanced + Java + Spring Boot + Docker

You now have production-grade Python skills + can build REST APIs in Java. GitHub should have 6+ projects. Time for PHASE 2: React!

PHASE 2

React Web Frontend

Months 4–6 | 11 Aug 2026 – 10 Nov 2026

HTML/CSS → JavaScript ES6+ → React Fundamentals → Full Stack Integration → Redux + Advanced

■ MONTH 4 — React Fundamentals | 11 Aug – 10 Sep 2026

You are a complete beginner in React. Month 4 starts from zero: HTML, CSS, JavaScript, then React. By end of month you'll have built 2 projects and understand how the modern web works.

Week	Topics	Project
Week 11 (11-17 Aug)	HTML5, CSS3, Flexbox, Grid, Responsive	Portfolio Website
Week 12 (18-24 Aug)	JavaScript ES6+: arrow fn, promises, fetch	JS DOM Projects
Week 13 (25-31 Aug)	React: JSX, components, props, useState	React Todo App
Week 14 (1-7 Sep)	useEffect, lists, forms, event handling	React Quiz App
Revision (8-10 Sep)	HTML/CSS/JS/React review	Polish portfolio

■ INTERVIEW CHECKPOINT 1 | End of Month 6 (10 Nov 2026)

- Python: Decorators, generators, async, OOP, design patterns
- Java: OOP, Spring Boot REST APIs, JPA, Spring Security
- React: Components, hooks, state management, API integration
- Docker: Containerisation, docker-compose
- DSA: Easy and Medium LeetCode problems (50+)
- System Design: Design a URL shortener, Design a Todo API

■ Mock Interview Activities

- Do 1 mock interview on Pramp (free) — pramp.com
- Record yourself answering 5 technical questions
- Practice system design whiteboard (pen + paper)
- Review STAR method answers for behavioral questions
- Solve 10 LeetCode problems in timed conditions (45 min each)
- Review company-specific interview patterns (Glassdoor, LeetCode Discuss)

■ MONTH 5 — React Intermediate + Full Stack | 11 Sep – 10 Oct 2026

React Router, useContext, custom hooks, API integration. Connect React to your Spring Boot backend. Build your first full-stack web app.

■ MONTH 6 — React Advanced + Redux + Tailwind | 11 Oct – 10 Nov 2026

Redux Toolkit for global state. Tailwind CSS for beautiful styling. Performance optimisation. Deploy full stack app to Vercel + AWS.

■ MILESTONE: PHASE 2 COMPLETE — React Full Stack

You can now build complete web applications. 3 full-stack projects on GitHub. Ready for Phase 3: Flutter Mobile Development!

PHASE 3

Flutter Mobile Development (iOS & Android)

Months 7–9 | 11 Nov 2026 – 10 Feb 2027

Dart Language → Flutter Basics → API Integration → Advanced Features → App Store Ready

■ MONTH 7 — Flutter Fundamentals + Dart | 11 Nov – 10 Dec 2026

Dart first, then Flutter. You'll set up Xcode (iOS simulator) and Android Studio. Build your first cross-platform app. Dart's syntax is similar to Java which you already know.

Week	Topics	Project
Week 19 (11-17 Nov)	Dart: types, functions, classes, null safety	Dart CLI Programs
Week 20 (18-24 Nov)	Flutter: widgets, hot reload, project structure	Counter App
Week 21 (25 Nov-1 Dec)	Stateful widgets, navigation, lists	Flutter Todo App
Week 22 (2-8 Dec)	Material Design, themes, responsive layout	Beautiful UI
Revision (9-10 Dec)	Dart + Flutter basics review	UI Polish

■ MONTH 8 — Flutter Intermediate + API Integration | 11 Dec 2026 – 10 Jan 2027

Provider state management. Connect Flutter app to Spring Boot API. Authentication flow. Real CRUD app that talks to your backend.

■ MONTH 9 — Flutter Advanced + App Store Ready | 11 Jan – 10 Feb 2027

Local storage with Hive. Push notifications with Firebase. Unit and widget testing. iOS and Android optimisation. Your app should be ready for deployment.

■ MILESTONE: PHASE 3 COMPLETE — Flutter Mobile Apps (iOS + Android)

You can now build cross-platform mobile apps. Connected to your Spring Boot backend. Ready for Phase 4: Cloud & DevOps!

PHASE 4

Months 10–12 | 11 Feb 2027 – 10 May 2027

AWS Core → Kubernetes → CI/CD Pipelines → EKS → AWS Certification

■ MONTH 10 — AWS Fundamentals | 11 Feb – 10 Mar 2027

EC2, S3, IAM, VPC, RDS — the core AWS services. Deploy your full stack app to AWS. Prepare for AWS Cloud Practitioner certification.

AWS Service	What it does	You will use it for
EC2	Virtual server in the cloud	Run Spring Boot application
S3	Object/file storage	Store React build, images, files
RDS	Managed relational database	PostgreSQL for your backend
IAM	Identity and access management	Secure access to all services
VPC	Private network in AWS	Isolate your app's network
CloudFront	Content Delivery Network	Serve React app globally fast
Lambda	Serverless function	Run code without managing servers
API Gateway	HTTP API routing	Route requests to Lambda/ECS
ECS/EKS	Container orchestration	Run Dockerized apps at scale
CloudWatch	Monitoring and logging	Watch your app's health
SQS/SNS	Message queues and notifications	Async communication between services

■ MONTH 11 — Kubernetes + AWS Certification | 11 Mar – 10 Apr 2027

Kubernetes architecture. Deploy to minikube locally. AWS Cloud Practitioner exam this month. Move to ECS for containerized deployment.

■ MILESTONE: ■ AWS CLOUD PRACTITIONER CERTIFICATION

Target: End of Month 11 (April 2027) | Cost: ~\$8,000 (~\$100)

■ MONTH 12 — CI/CD + EKS + Advanced DevOps | 11 Apr – 10 May 2027

GitHub Actions CI/CD pipeline. Deploy to AWS EKS (managed Kubernetes). Automated testing on every commit. Monitoring with CloudWatch.

■ INTERVIEW CHECKPOINT 2 | End of Month 12 (10 May 2027)

- AWS: EC2, S3, RDS, IAM, VPC, Lambda, ECS, EKS
- Docker: Dockerfile, docker-compose, multi-stage builds
- Kubernetes: pods, deployments, services, ingress
- CI/CD: GitHub Actions pipeline end-to-end
- System Design: Design scalable cloud architecture
- DSA: Medium LeetCode problems (100+ solved)
- LeetCode: Begin Hard problems

■ Mock Interview Activities

- Do 1 mock interview on Pramp (free) — pramp.com
- Record yourself answering 5 technical questions
- Practice system design whiteboard (pen + paper)
- Review STAR method answers for behavioral questions
- Solve 10 LeetCode problems in timed conditions (45 min each)
- Review company-specific interview patterns (Glassdoor, LeetCode Discuss)

■ MILESTONE: PHASE 4 COMPLETE — Cloud + DevOps

AWS certified. Full CI/CD pipeline. Apps deployed to EKS. Ready for Phase 5: Go + Microservices!

PHASE 5

Go Language + Microservices Architecture

Months 13–14 | 11 May 2027 – 10 Jul 2027

Go Basics → Concurrency → REST APIs → Microservices Patterns → Event-Driven Architecture

■ MONTH 13 — Go Fundamentals | 11 May – 10 Jun 2027

Go is Google's language. Simple, fast, built for concurrency. Goroutines are like lightweight async — you can run 100,000 of them. Perfect for microservices and high-throughput APIs.

Week	Topics	Project
Week 27 (11-17 May)	Go: syntax, types, functions, packages, errors	CLI calculator
Week 28 (18-24 May)	Structs, interfaces, methods, embedding	Go OOP patterns
Week 29 (25-31 May)	Goroutines, channels, select, WaitGroup	Concurrent downloader
Week 30 (1-7 Jun)	HTTP server, JSON, Gin framework, REST API	Go REST API
Revision (8-10 Jun)	Go review + Goroutine deep dive	Benchmark vs Python

■ MONTH 14 — Go Advanced + Microservices | 11 Jun – 10 Jul 2027

Go microservices communicating with Spring Boot services. Message queues with AWS SQS. API gateway pattern. Service discovery. This is enterprise architecture.

■ MILESTONE: PHASE 5 COMPLETE — Go + Microservices

Multi-language microservices running. Event-driven architecture implemented. Ready for Phase 6: AI/ML!

PHASE 6

AI / Machine Learning Integration

Months 15–16 | 11 Jul 2027 – 10 Sep 2027

NumPy/Pandas → ML Algorithms → Deep Learning → Model Deployment → FastAPI ML Service

■ MONTH 15 — ML Fundamentals | 11 Jul – 10 Aug 2027

Starting from zero in ML. NumPy, Pandas, Matplotlib for data work. scikit-learn for ML algorithms. Build and train your first model. Deploy it as a FastAPI endpoint.

Week	Topics	Project
Week 31 (11-17 Jul)	NumPy, Pandas, Matplotlib data analysis	Stock data analysis
Week 32 (18-24 Jul)	Linear regression, classification, evaluation	Price predictor
Week 33 (25-31 Jul)	Random forests, cross-validation, features	Improved predictor
Week 34 (1-7 Aug)	FastAPI + ML: serve model as REST API	ML API endpoint
Revision (8-10 Aug)	ML concepts + model evaluation review	Kaggle competition

■ MONTH 16 — Deep Learning + Deployment | 11 Aug – 10 Sep 2027

TensorFlow/Keras for deep learning. NLP basics with transformers. Deploy ML model to AWS Lambda. Integrate with React and Flutter apps.

■ MILESTONE: PHASE 6 COMPLETE — AI/ML Integration

ML model deployed as API. Integrated with web + mobile apps. Ready for Phase 7: AWS Advanced!

PHASE 7

AWS Advanced + Solutions Architect Certification

Month 17 | 11 Sep 2027 – 10 Oct 2027

DynamoDB → ElastiCache → Step Functions → Well-Architected Framework → SA Certification

Advanced AWS services. Solutions Architect Associate exam preparation. Design highly available, fault-tolerant, cost-optimized architectures. This certification adds ■5-10L to your salary.

Service/Topic	What you will learn
DynamoDB	NoSQL at scale: partition keys, GSI, streams, DynamoDB Accelerator
ElastiCache	Redis/Memcached for caching, session management, pub/sub
Step Functions	Serverless workflow orchestration, state machines
Cognito	User pools, identity pools, social login, MFA
Route 53	DNS management, routing policies, health checks, failover
CloudFormation	Infrastructure as code, stacks, templates, drift detection
Well-Architected	5 pillars: Security, Reliability, Performance, Cost, Operations
SA Exam Prep	Practice tests, white papers, case studies, architecture diagrams

■ MILESTONE: ■ AWS SOLUTIONS ARCHITECT ASSOCIATE CERTIFICATION

Target: End of Month 17 (October 2027) | Cost: ~■12,000 (~\$150)

PHASE 8

Capstone Project + Interview Grind + Job Applications

Month 18 | 11 Oct 2027 – 10 Nov 2027

SmartTrader Pro Platform + LeetCode Hard + System Design + Resume + Applications

■ CAPSTONE PROJECT: SmartTrader Pro Platform

The SmartTrader Pro is your showstopper project. One platform that uses ALL 9 skills. This is what you show in interviews. A recruiter should be able to clone it and run it in 5 minutes.

Layer	Technology	What it does
Web Frontend	React + Tailwind + Redux	Stock dashboard, portfolio, charts
Mobile App	Flutter (iOS + Android)	Same features on mobile, push alerts
Main Backend	Spring Boot (Java)	User management, portfolio, transactions
Microservice	Go (Gin)	Real-time stock data fetching, streaming
ML Service	Python FastAPI	Stock sentiment analysis, price prediction
Database (SQL)	PostgreSQL on AWS RDS	Users, portfolios, transactions
Database (NoSQL)	DynamoDB	Real-time price data (high throughput)
Cache	Redis (ElastiCache)	Session storage, frequently accessed data
Queue	AWS SQS	Async communication between services
Cloud	AWS EKS + ECS + Lambda	Scalable container orchestration
CDN	AWS CloudFront + S3	Serve React app globally
CI/CD	GitHub Actions	Auto-deploy on every push
Monitoring	AWS CloudWatch	Logs, metrics, alerts

■ INTERVIEW CHECKPOINT 3 | End of Month 18 (10 Nov 2027)

- Full tech stack: Python, Java, Go, React, Flutter, AWS, Docker, K8s, AI/ML
- System Design: Design Twitter, Design Netflix, Design payment gateway
- DSA: 150+ LeetCode (Easy + Medium + some Hard)
- Behavioral: STAR method, project stories, failure/success examples
- Resume: ATS-optimised, tailored for Full Stack / Backend roles
- Portfolio: 12+ GitHub projects, all with live deployments
- Mock interviews: 5+ on Pramp + peer practice

■ Mock Interview Activities

- Do 1 mock interview on Pramp (free) — [pramp.com](https://www.pramp.com)
- Record yourself answering 5 technical questions
- Practice system design whiteboard (pen + paper)
- Review STAR method answers for behavioral questions
- Solve 10 LeetCode problems in timed conditions (45 min each)
- Review company-specific interview patterns (Glassdoor, LeetCode Discuss)

■ EXPECTED OUTCOMES — Target Roles & Salaries

Role	Skills Used	Indian Market	Global Remote
------	-------------	---------------	---------------

Full Stack Engineer	React + Spring Boot + AWS	■25–45 LPA	\$100–140K
Mobile Engineer	Flutter + Spring Boot	■20–35 LPA	\$90–130K
Backend Engineer	Go + Java + Microservices	■30–50 LPA	\$120–160K
Cloud Engineer	AWS + K8s + CI/CD	■35–60 LPA	\$130–180K
Full Stack + Cloud	ALL SKILLS	■40–70 LPA ■	\$140–200K ■

■ MILESTONE: 18 MONTHS COMPLETE — YOU ARE READY!

9 skills mastered. 12+ GitHub projects. 2 AWS certifications. Full Stack + Mobile + Cloud engineer. Now go get that job! ■

■ TRADING — Personal Learning | Week 72

■■ This is SEPARATE from your daily 3-4hr tech learning. Do this in evenings (30-45 min).

- By Month 18 you should have: completed Zerodha Varsity (all modules)
- Opened a Demat account and started with ■5,000-10,000 paper money
- Learned: candlestick patterns, support/resistance, RSI, MACD
- Built a simple stock screener using your Python skills
- Understand: SIP, mutual funds, ETFs, index funds
- Journal: Track every paper trade — what worked, what didn't
- READ: The Intelligent Investor by Benjamin Graham (long-term mindset)
- WARNING: Do NOT put large amounts in day trading until 2+ years experience